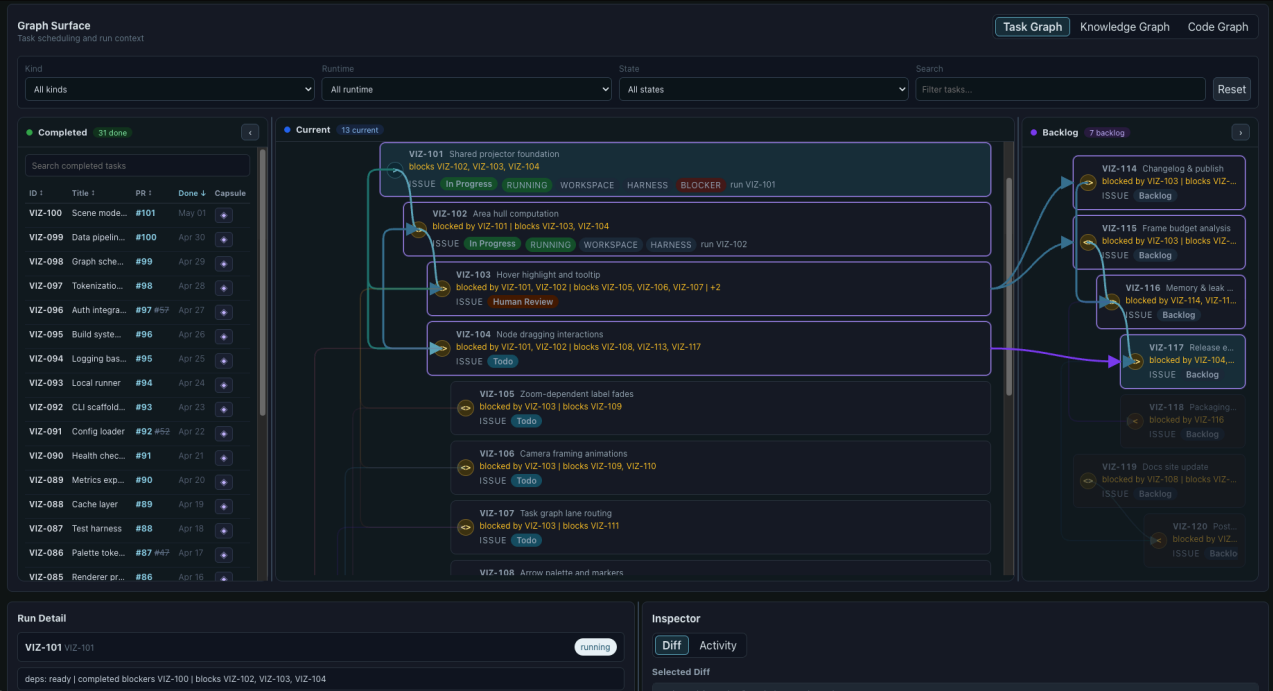


Orquestación de trabajo

Del spec a tareas ejecutables, revisión por agentes y memoria del proyecto.



OpenSymphony · captura de referencia del repositorio, con datos de ejemplo.

Conceptos y demostración en vivo

50 min

Curso de Parlina

Seis secciones. Recorrido por las portadas de las 23 lecciones, con ejemplos para el proyecto.

60 min

OpenSymphony

Instalación, planificación, publicación, despacho, landing y navegación por los grafos.

10 min

Preguntas

Elección de arnés, integración con el proyecto y decisiones de automatización.

La demo sigue una tarea nueva y retoma otra que ya pasó por varios ciclos de revisión.

Fundamentos y Taxonomía de la Ingeniería Agencial

Lección	Conceptos del curso	Aplicación en el proyecto
01 · Fronteras operativas	Modelo, arnés, meta-arnés y sistema de trabajo: responsabilidades y límites.	Identificar quién ejecuta herramientas, administra sesiones y coordina tareas.
02 · Clientes y hosts multiarnés	Topología host-cliente; Orca, Paseo y OpenChamber; instancias persistentes y worktrees paralelos.	Elegir dónde corren los agentes y desde qué interfaz se supervisan.
03 · Estado, persistencia y memoria	Estado de conversación, agente, repositorio y tracker; autoridad en base de datos, tracker o archivos.	Definir qué fuente gobierna cada estado y cómo reconciliarlo después de un fallo.

Al recuperar trabajo, distinguir el registro que gobierna el estado de los resúmenes o vistas derivados.

Patrones Arquitectónicos de Coordinación Agencial

Lección	Mecanismo de coordinación	Aplicación en el proyecto
04 · Ralph y variantes	Contexto fresco, estado en disco y ciclo Plan/Build; comprobaciones mecánicas que condicionan el avance.	Acotar entregas y definir validación, límites de iteración y condiciones de salida.
05 · Gas Town	Identidad persistente, Beads y Convoys; hooks, supervisión jerárquica y cola de integración Refinery.	Examinar aislamiento, capacidad y recuperación de trabajo concurrente como caso histórico.
06 · De GSD a GSD Pi	SQLite como autoridad durable, proyecciones Markdown y operaciones transaccionales; hitos, slices y tareas.	Comprobar idempotencia y recuperación ante fallos en el motor de trabajo.
07 · oh-my-openagent	IntentGate, agente principal y especialistas; enrutamiento multimodelo, contingencias y Team Mode en tmux.	Asignar modelos por tipo de trabajo y prever fallos de proveedor.

Resumen de las lecciones de Parlina. La actualización sobre Gas Town se presenta en la diapositiva 9.

Arneses de Desarrollo e Integración Programática

Lección	Conceptos del curso	Aplicación en el proyecto
08 · Codex app-server	JSON-RPC 2.0; Thread, Turn e Item; streaming, control de turnos y aprobaciones.	Diseñar el cliente y su contrato de ejecución, eventos y permisos.
09 · pi SDK y extensiones	AgentSession y AgentSessionRuntime; concurrencia, eventos, permisos y registro de herramientas.	Integrar el arnés en proceso y controlar la cola de mutación de archivos.
10 · oh-my-pi	Ejecución nativa; edición con Hashline; LSP, DAP y aislamiento de subagentes con pi-iso.	Analizar fallos de edición, comprobar semántica y depurar con herramientas estructurales.
11 · Claude Agent SDK, OpenHands y OpenCode	Topologías de integración, acoplamiento a proveedores y políticas de aislamiento.	Elegir SDK según despliegue, modelos, permisos y sandboxing.

La integración define cómo se inicia una ejecución, cómo se observa y qué controles limitan sus acciones.

OpenAI Symphony: Especificación Normativa y Motor Central

Lección	Conceptos del curso	Aplicación en el proyecto
12 · Contrato WORKFLOW.md	Front matter tipado, resolución de configuración, plantillas estrictas y recarga dinámica.	Validar el contrato y detener el despacho cuando su configuración es inválida.
13 · Workspaces	Persistencia por issue; validación de rutas, contención en la raíz y hooks de ciclo de vida.	Preparar y recuperar el entorno de cada tarea sin mezclar sus archivos.
14 · Política de despacho	Elegibilidad y orden estable; concurrencia, continuación de sesiones y reintentos con espera creciente.	Determinar qué puede ejecutarse ahora y cuándo volver a intentar.
15 · Reconciliación y seguridad	Detección de estancamiento, actualización desde el tracker y aislamiento de secretos.	Reconciliar la ejecución con el estado vigente y diagnosticar bloqueos.

OpenSymphony: Implementación Local-First en Rust y Extensiones

Lección	Conceptos del curso	Aplicación en el proyecto
16 · Arquitectura en Rust	Demonio y módulos internos; flujo de datos, despacho jerárquico y configuración.	Seguir el recorrido de un issue hasta su ejecución en el arnés.
17 · Linear y rutas de ejecución	Adaptador GraphQL, credenciales y cuotas; OpenHands Agent Server o Codex app-server.	Elegir y autenticar la ruta de ejecución; recuperar sesiones de depuración.
18 · Grafos y memoria	Grafo de tareas, cápsulas Markdown, índice DuckDB y AST con Tree-sitter.	Capturar conocimiento de tareas completadas y recuperar contexto para las siguientes.
19 · Supervisión operativa	FrankenTUI y desktop; doctor, diffs, grafos, debug y rehidratación.	Inspeccionar un bloqueo, comprobar el entorno y retomar una sesión.

Gobernanza, Verificación Mecánica y Evaluación Empírica

Lección	Conceptos del curso	Aplicación en el proyecto
20 · Planificación y publicación	Todo el alcance, task-package.yaml, Definition of Ready, grafo acíclico y publicación idempotente.	Comprobar cobertura y dependencias antes de publicar en Linear.
21 · Calidad y cierre	Pruebas, controles automatizados, autoridad de transición, presupuestos y límites de reintento.	Exigir evidencia para cerrar una tarea y detectar falta de progreso.
22 · EBO y Behavior Atlas	Ejecución, captura y resolución; trayectorias del arnés, ontología de comportamiento y evaluación.	Inspeccionar evidencia capturada y analizar anomalías de comportamiento.
23 · Mejora de arneses	exoharness: ejecutor efímero y estado durable. Prime Agent: arnés continuo y /refine.	Observar, diagnosticar, refinar y validar cambios con un estado previo recuperable.

En exoharness, el ejecutor propone cambios; el arnés conserva la autoridad sobre estado, recursos y controles.

Gas Town: coordinación y confiabilidad

La propuesta de Yegge

Trabajo representado como grafo, con roles, memoria, autoridad, revisión y recuperación.

Su giro hacia Wheelhouse plantea una orquestación ligada a las necesidades del producto.

Síntesis aportada por el instructor de [The Shape of Things to Come](#).

La crítica sobre resultados

El [comentario citado en AI News](#) interpreta que Yegge no logró construir algo con Gas Town y cuestiona la confiabilidad de estos orquestadores.

Es una crítica reportada; no aporta aquí una comparación experimental entre sistemas.

Para evaluar un orquestador: seguir tareas hasta su integración y registrar dónde necesitaron corrección o intervención.

Qué aporta EBO hoy

Eventos

agente / arnés



Ontología

comportamiento



Evaluación

criterios y evidencia

Alcance actual

Observabilidad de eventos de agente y arnés, con una capa de ontología de comportamiento y su evaluación.

Permite analizar el comportamiento capturado; la evidencia disponible depende de la integración.

Posible extensión futura

Vincular esas observaciones con tareas, revisiones y resultados de integración.

Requeriría instrumentación adicional para investigar dónde falla la coordinación. No forma parte del alcance actual.

Engineering Behavior Observatory

Del spec al landing

- | | |
|--------|---|
| 10 min | Instalar y configurar OpenSymphony en un repo de demostración. |
| 15 min | Descomponer el alcance, revisar el paquete y publicar tareas en Linear. |
| 10 min | Correr el orquestador y observar un despacho. |
| 10 min | Recorrer revisiones previas e integrar un PR ya preparado. |
| 15 min | Explorar tareas, cápsulas de memoria y código en la app desktop. |

La tarea despachada puede seguir ejecutándose mientras examinamos el PR ya revisado.

Conectar el repositorio, Linear y el arnés

Terminal del repositorio

```
cargo install opensymphony  
opensymphony init  
  
opensymphony run  
  
# En otra terminal  
opensymphony desktop
```

Comandos de la [documentación de OpenSymphony](#).

Configuración que revisaremos

Acceso a Linear y al repositorio GitHub.

Arnés autenticado, modelo y permisos.

Proyecto observado, estados de trabajo y rama de destino.

WORKFLOW.md y config.yaml.

Arrancar el orquestador después de revisar el proyecto y la política de despacho.

Del alcance completo a las tareas de Linear



Skill	Resultado que inspeccionamos
<code>create-implementation-plan</code>	Manifest, hitos y un archivo por tarea; aceptación, contexto y validación.
<code>convert-tasks-to-linear</code>	Validación, dry-run y publicación del paquete revisado.
<code>linear</code>	Consulta y actualización de issues, dependencias y estados.

Cada requisito del alcance debe corresponder a tareas publicadas o a evidencia de implementación existente.

Qué recibe el agente

Tarea	Objetivo, alcance, criterios de aceptación y dependencias.
Repositorio	Workspace, instrucciones, código y comando de validación.
Contexto	Decisiones previas y memoria relevante para ese trabajo.
Agent Workpad	Plan, avances, hallazgos, pruebas y bloqueos registrados durante la ejecución.

En la app: seleccionar el issue, abrir su ejecución y comprobar el contexto y los primeros eventos.

Si la tarea no se despacha, revisar dependencias, estado del issue y configuración antes de reintentar.

Entrega un cambio que se pueda revisar



En el PR preparado	Qué comprobar
Ciclos anteriores	Un hallazgo, la corrección y la verificación posterior.
Revisión vigente	Que cubre el commit que se va a integrar.
Resultado	Aceptación cubierta, CI y ausencia de hallazgos pendientes que bloqueen el merge.

La revisión debe corresponder al commit que se va a integrar.

Integrar el PR y actualizar el trabajo

PR preparado

revisión y CI



Merge

rama de destino



Linear

estado y enlace



Memoria

captura y consulta

En esta demostración

Comprobar el commit revisado, hacer merge y verificar el resultado en GitHub.

Actualizar Linear según el workflow y comprobar la captura de memoria.

Política del proyecto

El merge y el cambio de estado se pueden automatizar con condiciones explícitas.

La intervención humana permite decidir alcance, aceptar riesgos y conservar comprensión del código.

La política del proyecto define las condiciones de merge y de cierre en Linear.

Dependencias, ejecución y resultado

Graph Surface

Task scheduling and run context

Task GraphKnowledge GraphCode Graph

Kind

All kinds

Runtime

All runtime

State

All states

Search

Filter tasks...

Reset

Completed31 done

Search completed tasks

ID ↓	Title ↓	PR ↓	Done ↓	Capsule
VIZ-100	Scene mode...	#101	May 01	+
VIZ-099	Data pipelin...	#100	Apr 30	+
VIZ-098	Graph sche...	#99	Apr 29	+
VIZ-097	Tokenizatio...	#98	Apr 28	+
VIZ-096	Auth integra...	#97 #57	Apr 27	+
VIZ-095	Build syste...	#96	Apr 26	+
VIZ-094	Logging bas...	#95	Apr 25	+
VIZ-093	Local runner	#94	Apr 24	+
VIZ-092	CLI scaffold...	#93	Apr 23	+
VIZ-091	Config loader	#92 #52	Apr 22	+
VIZ-090	Health chec...	#91	Apr 21	+
VIZ-089	Metrics exp...	#90	Apr 20	+
VIZ-088	Cache layer	#89	Apr 19	+
VIZ-087	Test harness	#88	Apr 18	+
VIZ-086	Palette toke...	#87 #47	Apr 17	+
VIZ-085	Renderer pr...	#86	Apr 16	+

Current13 current

VIZ-101

Shared projector foundation

blocks VIZ-102, VIZ-103, VIZ-104

ISSUE In Progress RUNNING WORKSPACE HARNESS BLOCKER run VIZ-101

VIZ-102

Area hull computation

blocked by VIZ-101 | blocks VIZ-103, VIZ-104

ISSUE In Progress RUNNING WORKSPACE HARNESS run VIZ-102

VIZ-103

Hover highlight and tooltip

blocked by VIZ-101, VIZ-102 | blocks VIZ-105, VIZ-106, VIZ-107 | +2

ISSUE Human Review

VIZ-104

Node dragging interactions

blocked by VIZ-101, VIZ-102 | blocks VIZ-108, VIZ-113, VIZ-117

ISSUE Todo

VIZ-105

Zoom-dependent label fades

blocked by VIZ-103 | blocks VIZ-109

ISSUE Todo

VIZ-106

Camera framing animations

blocked by VIZ-103 | blocks VIZ-109, VIZ-110

ISSUE Todo

VIZ-107

Task graph lane routing

blocked by VIZ-103 | blocks VIZ-111

ISSUE Todo

VIZ-108

Arrow palette and markers

Backlog7 backlog

VIZ-114

Changelog & publish

blocked by VIZ-103 | blocks VIZ-...

ISSUE Backlog

VIZ-115

Frame budget analysis

blocked by VIZ-103 | blocks VIZ-...

ISSUE Backlog

VIZ-116

Memory & leak ...

blocked by VIZ-114, VIZ-11...

ISSUE Backlog

VIZ-117

Release e...

blocked by VIZ-104,...

ISSUE Backlog

VIZ-118

Packaging...

blocked by VIZ-116

ISSUE Backlog

VIZ-119

Docs site update

blocked by VIZ-108 | blocks VIZ-...

ISSUE Backlog

VIZ-120

Post...

blocked by VIZ-...

ISSUE Backlo

Run Detail

VIZ-101 VIZ-101

running

deps: ready | completed blockers VIZ-100 | blocks VIZ-102, VIZ-103, VIZ-104

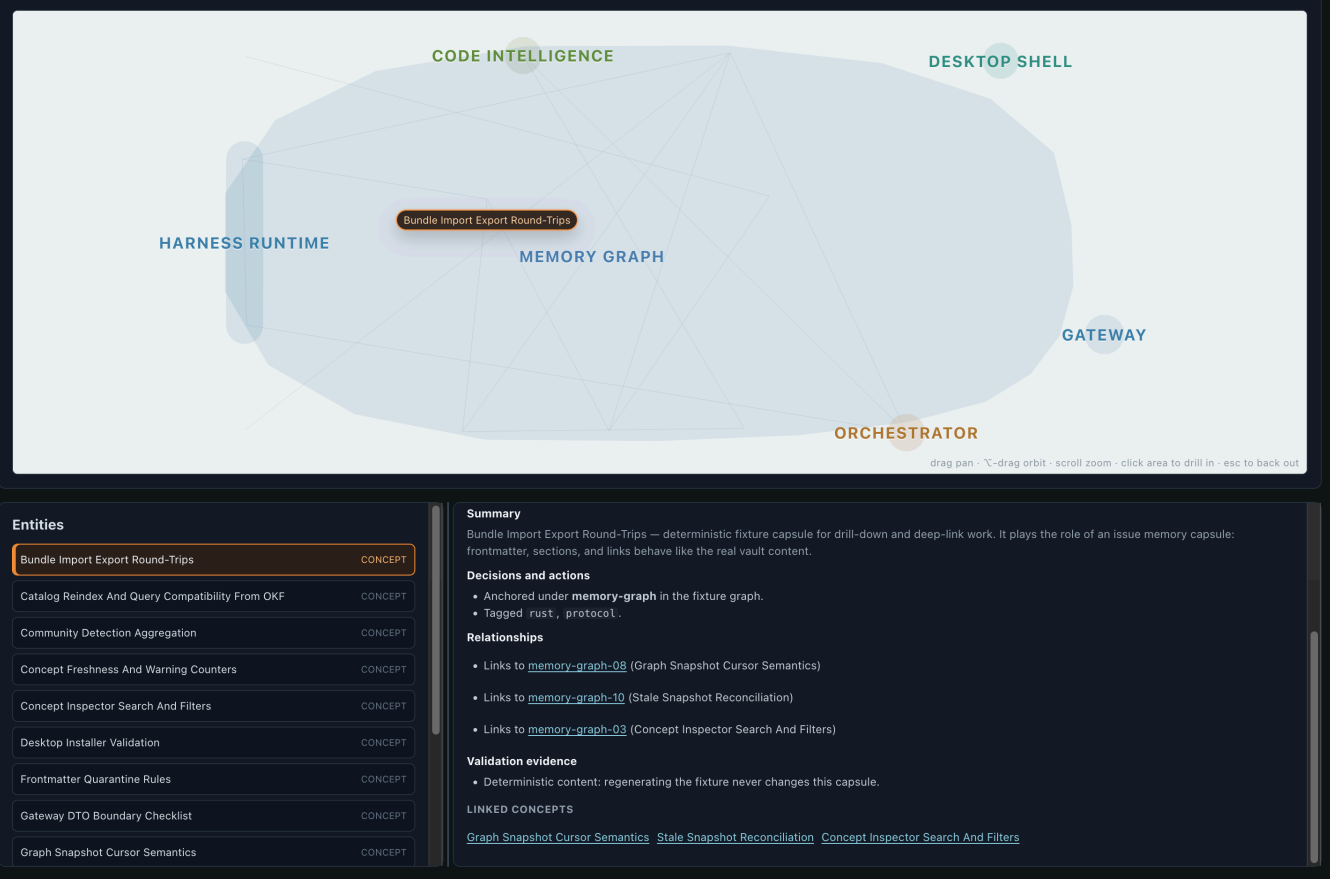
Inspector

DiffActivity

Selected Diff

Captura de referencia con datos de ejemplo. En vivo: seleccionar una tarea, seguir sus dependencias y abrir ejecución y PR.

Del Agent Workpad al contexto de otra tarea



Durante el trabajo

El Agent Workpad conserva decisiones, resultados y asuntos pendientes.

Después del cierre

La captura conserva conocimiento de la tarea y del PR en cápsulas durables.

En la siguiente tarea

Consultar memoria relacionada y comprobar sus fuentes antes de usarla como contexto.

Captura de referencia con una cápsula de ejemplo. En vivo: abrir una cápsula real y seguir un enlace a su fuente.

Localizar símbolos y comprobar el código vigente

Archivos

repositorio / workspace



Tree-sitter

estructura sintáctica



Grafo

símbolos y referencias



Contexto

fuentes acotadas

En la app

Buscar un símbolo, abrir sus relaciones y navegar al archivo correspondiente.

Comparar con el diff de una ejecución cuando esté disponible.

Code Intelligence: búsqueda indexada y comprobación del AST

En el trabajo del agente

Usar el índice para encontrar código relevante y comprobar el AST del workspace antes de editar.

La búsqueda estructural orienta la modificación; las pruebas comprueban su efecto.

Probar orquestación con el plan de E6

Usa OpenSymphony para recorrer una tarea de tu proyecto desde el despacho hasta la integración y la memoria.

Con cualquier arnés u orquestador	Qué conservar
Despacho	Tarea identificable, dependencias y condiciones de inicio.
Contexto	Instrucciones, código y decisiones relevantes.
Continuidad	Notas de trabajo, evidencia y memoria consultable.
Integración	Revisión por agentes, validación y política de cierre.

Un orquestador propio se justifica cuando necesitas reglas o integraciones que los existentes no cubren y puedes mantener su operación.

[OpenSymphony](#) · [Recursos de E6 y E7](#) · [Preparación del plan completo](#)